

Signhify — Local IDE Development Guide

Continue building Signhify in any local AI-native IDE (Google **Antigravity**, **Cursor**, **Windsurf**, **Kiro**, **Gemini CLI**, **Kilo CLI**, **Claude Code**, **Aider**, **Zed**, or **plain VS Code**) while keeping deployments shipping through **Lovable**.

>

Last updated: June 8, 2026 · Maintained by Piyush Raj Singh ·

hello@signhify.online

This project is bidirectionally synced with GitHub. Anything you push to the default branch is picked up by Lovable within seconds and becomes part of the next preview / publish. Anything you change in Lovable is committed straight back to GitHub. That is the whole contract — everything below is the practical detail to make it boringly reliable.

1. One-time setup

1.1 Prerequisites

Tool	Min version	Why
Node.js	20.x LTS	Required by Vite 7 + TanStack Start
Bun	1.1+	Package manager + script runner (bun.lockb is source of truth)
Git	2.40+	GitHub sync
GitHub account	—	Bidirectional sync with Lovable
Lovable workspace access	—	To publish

Do not swap Bun for npm/pnpm/yarn. The **bun.lockb** file is the source of truth and Lovable's GitHub sync expects it. Mixing package managers produces lockfile drift and breaks the auto-deploy.

1.2 Connect Lovable ↔ GitHub (do this once)

In Lovable: **Plus (+) menu** → **GitHub** → **Connect project** → **Create Repository**.

Pick the org/account you want. Lovable creates the repo and pushes the current codebase.

1.3 Clone locally

```
git clone git@github.com:<your-org>/signhify.git
cd signhify
bun install
cp .env.example .env    # if present, otherwise create .env (see §2)
bun run dev
```

The dev server runs on **http://localhost:3000**. Hot reload, SSR, and TanStack

Router file-based routing all work the same as inside Lovable.

1.4 Full copy/paste cheat-sheet

Every command you need, in the order you need them. Copy the whole block into a fresh terminal — it is idempotent.

```
# 0. prerequisites (macOS / Linux). Windows: use WSL2.
curl -fsSL https://bun.sh/install | bash          # install Bun
node -v && bun -v && git --version                # sanity check

# 1. clone + install
git clone git@github.com:<your-org>/signhify.git
cd signhify
bun install                                         # uses bun.lockb – do NOT swap for npm/pnpm/yarn

# 2. env – create .env from the template below (see §2 for full variable list)
cat > .env <<'EOF'
VITE_SUPABASE_URL=https://<project>.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=<anon-key>
VITE_SUPABASE_PROJECT_ID=<project-ref>
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_PUBLISHABLE_KEY=<anon-key>
SUPABASE_SERVICE_ROLE_KEY=<service-role-key>    # server-only, never ship to client
LOVABLE_API_KEY=                                # optional, only if you call the AI gateway locally
EOF

# 3. run the dev server (http://localhost:3000, SSR + HMR)
bun run dev

# 4. production build + local preview
bun run build                                     # outputs to .output/
bun run start                                    # or: bun run preview    # serves the built app

# 5. quality gates (run before every push)
bun run lint
```

```
bun run format
bun run prepublish:check # Playwright smoke + HTML diff

# 6. regenerate the local-dev-guide PDF after editing the .md
bun run guide:pdf

# 7. ship it
git add -A && git commit -m "feat: <what changed>" && git push
# Lovable auto-pulls the commit and rebuilds the preview within seconds.
```

`bun run start` is an alias for the production preview. Use it to sanity-check the bundled output before pushing. `bun run dev` is the only command you need for day-to-day iteration.

2. Environment variables

Two scopes. Never mix them.

Browser / build-time (VITE_)*

Safe to bundle. Put in local `.env`:

```
VITE_SUPABASE_URL=https://<project>.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=eyJ...
```

Server / runtime (process.env.)*

Secrets. Read inside `.handler()` of server functions only.

```
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_PUBLISHABLE_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ... # never commit, never expose to client
LOVABLE_API_KEY=... # optional, for AI gateway
```

On Lovable Cloud the runtime secrets are injected automatically — your local `.env` is only for `bun run dev` on your machine. To rotate or add a runtime secret used in production, do it from the ****Lovable → Cloud → Secrets**** panel (not from GitHub).

2.1 Where to find each value

Variable	Where to get it
VITE_SUPABASE_URL / SUPABASE_URL	Supabase Dashboard → Project Settings → API → Project URL

VITE_SUPABASE_PROJECT_ID	The subdomain of that URL (e.g. <code>ngeuarvpkxupxeeuzuow</code>)
VITE_SUPABASE_PUBLISHABLE_KEY / SUPABASE_PUBLISHABLE_KEY	Same page → anon / public key. Safe in the browser.
SUPABASE_SERVICE_ROLE_KEY	Same page → service_role key. Server-only . Never ship to the client, never commit.
LOVABLE_API_KEY	Lovable → Project → Cloud → AI Gateway . Only needed if you call the AI Gateway from your local dev server.

2.2 Make sure the deployed Worker has the same secrets

`/publish` writes an audit row using the **service role** key. If the Worker on Lovable Cloud is missing `SUPABASE_SERVICE_ROLE_KEY` you'll see:

```
`Could not record audit: Missing Supabase environment variable(s):
SUPABASE_SERVICE_ROLE_KEY. Connect Supabase in Lovable Cloud.`
```

Fix it once and the auto-retry on `/publish` will record the audit on the next attempt.

Open Lovable → **Project** → **Cloud** → **Secrets**.

Confirm all four are present and not empty:

- `SUPABASE_URL`
- `SUPABASE_PUBLISHABLE_KEY`
- `SUPABASE_SERVICE_ROLE_KEY`
- `LOVABLE_API_KEY` (only if you use the AI Gateway)

If any are missing, click **Add secret**, paste the value from the Supabase Dashboard (§2.1), and **Save**.

Re-deploy so the new secret reaches the Worker. Either:

- push any commit to `main` (frontend changes auto-deploy on push; secret-only changes still need a deploy to refresh the Worker), **or**

- in Lovable, click **Publish** → **Update** (frontend) to force a rebuild that picks up the new secret bindings.

Open `/publish` in preview. The **Supabase connectivity** card should turn green (`hasUrl`, `hasServiceRole`, `adminProbe` all ✓). If it doesn't, click **Re-check** and watch the auto-retry arm itself.

Run `bun run prepublish:check` locally one more time, then publish.

The local `.env` and Lovable Cloud Secrets are **two separate stores**.

Updating `.env` only affects `bun run dev` on your machine. Production reads from Lovable Cloud Secrets — always update both.

3. IDE-specific setup

Pick whichever you use. All of them work because they only need (a) the filesystem and (b) `git`.

3.1 Cursor / Windsurf / Zed / VS Code

Open the cloned folder. Recommended extensions:

- Biome / Prettier (formatting already configured via `.prettierrc`)
- ESLint
- Tailwind CSS IntelliSense
- TanStack Router (optional — gives route-tree hints)

Cursor/Windsurf agents: point them at `CONTRIBUTING.md` and `DEPLOY.md` as "always-attached" context so they respect the project's conventions.

3.2 Google Antigravity

File → **Open Workspace** on the cloned repo.

In the Agent panel, add `CONTRIBUTING.md`, `DEPLOY.md`, and this guide to *Knowledge*.

Configure the **Run Task** integration to use `bun run dev` and `bun run prepublish:check`.

Antigravity's Browser Use mode targets `http://localhost:3000` — keep

the dev server running while you iterate.

3.3 Kiro

`kiro init` inside the repo to register the spec workspace.

Create a spec at `.kiro/specs/feature-name/` for each new feature; Kiro will scaffold and edit routes under `src/routes/` following the TanStack Start conventions documented in `src/routes/README.md`.

Run `kiro hooks add prepublish` so Kiro runs `bun run prepublish:check` before any push to `main`.

3.4 Gemini CLI

```
npm install -g @google/gemini-cli
gemini auth login
cd signhify
gemini                                # interactive session
# or one-shot
gemini -p "add a pricing FAQ section to /pricing"
```

Add a **GEMINI.md** at the repo root if you want to override the default context window. The CLI reads **CONTRIBUTING.md** automatically.

3.5 Kilo CLI

```
brew install kilo-cli          # or curl install per docs
kilo init
kilo run "implement <task>"
```

Kilo respects `.kilo/config.yaml`. Add `prepublish:check` as the pre-commit guard so failed Playwright runs block the commit.

3.6 Claude Code / Aider / OpenCode

Drop in at the repo root and go. They all use `git` for change tracking and will respect `bun.lockb`. For Aider:

```
aider --model claude-3-5-sonnet --read CONTRIBUTING.md DEPLOY.md
```

4. Day-to-day workflow

[illegible]

[illegible]

4.1 Branching

- Work on short-lived feature branches: `feat/<scope>` or `fix/<scope>`.
- Open a PR into `main`. Lovable previews track `main` by default; enable

GitHub Branch Switching in **Account** → **Labs** if you need per-branch previews.

- Squash-merge to keep the Lovable timeline readable.

4.2 Things you must never hand-edit

These are regenerated and will cause merge conflicts or runtime breakage:

- `src/routeTree.gen.ts`
- `src/integrations/supabase/types.ts`
- `src/integrations/supabase/client.ts`, `client.server.ts`,
`auth-middleware.ts`, `auth-attacher.ts`
- `bun.lockb` (let `bun install` manage it)

4.3 Database changes

Always go through a migration file:

```
# create the file
touch supabase/migrations/$(date +%Y%m%d%H%M%S)_<name>.sql
# write SQL – remember GRANTS for any new public-schema table
```

Lovable picks up new migrations automatically on deploy. Never run ad-hoc SQL against production from your local shell.

4.4 Server functions vs. server routes

- `createServerFn` (TanStack Start) for internal RPC — files in `src/lib/*.functions.ts`.
- `createFileRoute` with a `server` block for webhooks/public APIs — files in `src/routes/api/public/*`.

Do **not** add Supabase Edge Functions for new work — they're being phased out in favour of TanStack server functions.

5. Pre-publish checklist (mandatory)

Codified in `CONTRIBUTING.md` §8. The short version:

```
bun run prepublish:check
```

This runs the Playwright smoke + HTML-diff suite against the latest preview. It must pass before you publish from Lovable. If you publish without it, the `/publish` audit row will not be created and the gate will block subsequent ships.

Manual gates in addition to the script:

- [] Preview health: `/api/public/health` returns `{ok:true}`
- [] Marketplace diff is intentional (compare against `src/lib/marketplace.ts`)
- [] SEO: every changed route has unique title, description, OG + Twitter tags
- [] Security scan clean (no unresolved critical findings)
- [] Sync hygiene: `git status` clean, `bun.lockb` unchanged unless deps changed

6. Publishing

You have two equivalent paths:

From Lovable — open the project, click **Publish** (top-right on desktop, bottom-right on mobile in preview). Frontend changes deploy on click; backend changes (migrations, server functions) deploy automatically as soon as they land on `main`.

From your IDE — push to `main`, then ask the user (or yourself in Lovable chat) to publish. The agent walks through the Pre-Publish checklist and only then calls the publish tool.

Stable URLs:

Purpose	URL
Production (Lovable)	<code>https://signhify.lovable.app</code>
Production (custom)	<code>https://signhify.online</code>

Preview (stable)	<code>https://project--<project-id>-dev.lovable.app</code>
Production (stable)	<code>https://project--<project-id>.lovable.app</code>

Use the stable `project--...` URLs for webhooks and cron — they survive project renames.

7. Self-hosting elsewhere (optional)

Even after wiring local dev, you can still host the build outside Lovable. The repo targets Cloudflare Workers / Vercel Edge / Netlify without modification. See [DEPLOY.md](#) for variables and adapters.

```
bun run build          # produces .output/
bun scripts/check-deploy.mjs https://your-host.example
```

8. Troubleshooting

Symptom	Likely cause	Fix
Failed to resolve <code>import "@/..."</code> after <code>git pull</code>	New file referenced before <code>bun install</code>	Run <code>bun install</code>
Missing Supabase <code>env var(s)</code> in console	<code>.env</code> not loaded	Restart <code>bun run dev</code> ; verify variable names
<code>Unauthorized</code> from a <code>createServerFn</code> in SSR	Protected fn called from a public-route loader	Call it from the component via <code>useServerFn</code> (see CONTRIBUTING.md)
Lovable preview shows old code	Push didn't land on <code>main</code>	Check GitHub Actions; force a re-sync from Lovable's GitHub panel
<code>bun.lockb</code> conflict	Two contributors edited deps in parallel	Delete <code>bun.lockb</code> , re-run <code>bun install</code> , commit

Playwright smoke fails locally	Wrong base URL	<pre>PLAYWRIGHT_BASE_URL=https://id-preview--<id>.lovable.app bun run prepublish:check</pre>
<code>__dirname</code> is not defined in production only	Used a Node-only package in a server fn	Replace with a Workers-compatible alternative (see server-runtime docs)

9. Golden rules

GitHub is the source of truth. Both Lovable and your IDE are editors on top of it.

Never hand-edit generated files. They will be overwritten and merge-conflict at the worst moment.

Secrets live in Lovable Cloud, not in `.env` for production.

Pre-publish check is non-optional. It exists because we have shipped broken marketplace diffs before.

Small PRs, fast merges. The Lovable preview rebuild is fast — use it as a real review surface.

Welcome to the codebase. Ship something.